COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	September 13, 2026	**DATE SUBMITTED:**	September 13, 2026
NAME:	Garbosa, Isaiah Benedict R.	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 8.1 Sorting Algorithms Pt2

1. PROCEDURE OUTPUT

-

Code

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sorting.h"

const int size = 100;

int main(){
    int arr[size];

    srand(time(0));

    for(int i = 0; i< size; i++){
        arr[i] = rand() % 10+1;
    }

    std::cout<<"Original Array: ";
    displayArray(arr, size);

    std::cout<<std::endl;
```

Output

```
Original Array: 9 7 8 7 3 6 3 1 3 7 1 5 10 1 7 6 2 3 9 3 8 6 3 8 10 5 3 10 5 7 9
 3 4 8 6 8 8 10 1 10 7 10 4 3 8 7 1 9 4 9 1 6 1 6 9 10 1 9 2 7 2 9 4 7 2 9 3 4 2
 9 3 6 2 5 2 1 1 10 1 7 2 2 5 8 6 10 5 9 9 3 1 9 9 7 10 6 4 5 8 8
```

Observation
The code uses cstdlib and ctime to generate random numbers. It generates 100 random numbers and ranging from 1 to 10.

Table 8-1. Array of Values for Sort Algorithm Testing

Code

```
template <typename T>
void shellSort(T arr[], int size){

    for(int gap = size / 2; gap > 0; gap /= 2){
        for (int i = gap; i < size; i++){
            T temp = arr[i];
            int j = i;
            while(j >= gap && arr[j - gap] > temp){
                arr[j] = arr[j - gap];
                j = j - gap;
            }
            arr[j] = temp;
        }
    }
}
```

```
std::cout<<"\nShell Sort: ";
shellSort(arr, size);
displayArray(arr, size);
```

Output
Shell Sort: 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 5 5 5 5 5 5 6 6 6 6 6 6 6 6 6 7 7 7 7 7 7 7 7 7 7 8 8 8 8 8 8 8 8 8 9 9 9 9 9 9 9 9 9 9 9 9 9 9 10 10 10 10 10 10 10 10 10 10
Observation
The code works by getting a gap. We get the gap by dividing the size of the array into 2. It then compares elements between the gap and switches them if needed. The gap will continue to decrease and compare the elements between the new gaps. Once the gap is 1, the rest of the array is then sorted to their proper positions. The time complexity of the shell sorts if O(nlogn) in best case and O(n²) in the worst case. The best case happens if the array is already sorted and the worst case is when both the for loop goes through each of the elements/

Table 8-2. Shell Sort Technique

Code

```
template <typename T>
void mergeSort(T arr[], int left, int right){

    if (left<right){
        int middle = (left + right) / 2;
        mergeSort(arr, left, middle);
        mergeSort(arr, middle+1, right);
        merge(arr, left, middle, right);
    }
}
```

```cpp
template<typename T>
void merge(T arr[], int left, int middle, int right){
    int i = left;
    int j = middle +1;
    int k = 0;

    int size = right - left + 1;
    T* temp = new T[size];

    while (i <= middle && j <= right){
        if(arr[i] <= arr[j]){
            temp[k] = arr[i];
            i++;
        }
        else{
            temp[k] = arr[j];
            j++;
        }
        k++;
    }

    while (i<=middle){
        temp[k] = arr[i];
        i++;
        k++;
    }

    while(j <= right){
        temp[k] = arr[j];
        j++;
        k++;
    }

    for (int x = 0; x < size; x++){
        arr[left + x] = temp[x];
    }

    delete[] temp;
}
```

```
std::cout<<"\nMerge Sort: ";
mergeSort(arr, 0, size-1);
displayArray(arr, size);
```

Output

```
Merge Sort: 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 4
4 4 4 4 4 5 5 5 5 5 5 6 6 6 6 6 6 6 6 6 7 7 7 7 7 7 7 7 7 7 8 8 8 8 8 8 8 8
8 8 9 9 9 9 9 9 9 9 9 9 9 9 9 10 10 10 10 10 10 10 10 10 10
```

Observation

The code works by getting the middle of the array, then separating the array into two subarrays. It divides the array into left and right. Each of the subarrays are sorted first by comparing elements. After, the elements are then merged back together by comparing an element from the left subarray and an element from the right subarray, organizing the array. The time complexity of the O(nlogn) for both the best case and worst case since it uses a divide and conquer approach.

Table 8-3. Merge Sort Algorithm

Code

```cpp
template <typename T>
int partition(T arr[], int low, int high){
    T pivot = arr[high];
    int i = low - 1;

    for(int j = low; j<high; j++){

        if(arr[j] < pivot){
            i++;

            T temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    T temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

template <typename T>
void quickSort(T arr[], int low,  int high){

    if (low < high){

        int pivot = partition(arr, low, high);
        quickSort(arr, low, pivot - 1);
        quickSort(arr, pivot+1, high);
    }
}
```

```
std::cout<<"\nQuick Sort: ";
mergeSort(arr, 0, size-1);
displayArray(arr, size);
```

Output

```
Quick Sort: 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 4
4 4 4 4 4 5 5 5 5 5 5 6 6 6 6 6 6 6 6 6 7 7 7 7 7 7 7 7 7 8 8 8 8 8 8 8
8 8 9 9 9 9 9 9 9 9 9 9 9 9 9 10 10 10 10 10 10 10 10 10 10
```

Observation

The code works by using a pivot to divide the array into smaller parts. The pivot is typically set on the last element. Then, two variables are created, i and j. Based on the conditions, the elements are arranged based on whether they are smaller or greater than the pivot. The process repeats until all the elements are sorted. The time complexity for quick sort is O(n log n) for its best case and O(n^2) for its worst case.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

- Yes, the left and right sublists from the partition in quick sort can be sorted using other sorting algorithms. This can be done since the partition only separates the elements, it does not completely sort them. So, a different sorting method can be used. For an example we have an array, the left sublist is [3, 7, 2, 4, 6] and the pivot is 8 and the right sublist is [9]. If the left side contains values smaller than 8, it can be sorted using a different algorithm, as long as it correctly sorts the values

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

- For this given array, quick sort can be the best choice for sorting the array with the fastest time performance. Its average time complexity is O(n log n) which generally performs fewer operations compared with merge sort. Merge sort and quick sort have O(n log n) for their time complexity since for merge sort, the array is repeatedly divided into two halves. At each level, elements are processed which take O(n). For quick sort, it depends on the pivot. If the pivot divides the array evenly, there are about log N levels and each level required O(n).

Conclusion
For this activity, I learned more ways to sort an array. I learned three new methods for sorting. First is the shell sort which compares elements using a gap and switches them if they are not sorted. The gap gets smaller and smaller until the gap is 1, where the array's elements are now closer and can be sorted using insertion sort. Next is the merge sort which pretty much divides an array into sublists. Each sublist is then sorted. Once the sublists are sorted, they are then merged back together by comparing elements from each sublist and comparing which elements are smaller or higher. Finally, the quick sort utilizes a pivot to compare elements on the array. Overall, I was able to learn and apply the different sorting algorithms. I would like to practice more and find out how we can apply these sorting algorithms in a more real-life situation.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **n/a**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **n/a**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **n/a**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.